Name :Sara Gamal Taha

Group:SE3

Lab 4

(1) Screenshots of successful API calls and failure simulation

```
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.22631.6936]
(c) Microsoft Corporation. All rights reserved.

C:\week4_lab>curl http://localhost:5011/products/1
{"id":1,"name":"Laptop","price":1000}

C:\week4_lab>
```

```
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.22631.6936]
(c) Microsoft Corporation. All rights reserved.

C:\week4_lab>curl http://localhost:5011/products/1
{"id":1,"name":"Laptop","price":1000}

C:\week4_lab>curl -X POST http://localhost:5003/orders -H "Content-Type: application/json" -d "{\"product_id\":1,\"quantity\":2}"
{"message":"Order created","product":{"id":1,"name":"Laptop","price":1000},"quantity":2}

C:\week4_lab>
```

```
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.22631.6936]
(c) Microsoft Corporation. All rights reserved.

C:\week4_lab>curl http://localhost:5011/products/1
{"id":1,"name":"Laptop","price":1000}

C:\week4_lab>curl -X POST http://localhost:5003/orders -H "Content-Type: application/json" -d "{\"product_id\":1,\"quantity\":2}"
{"message":"Order created","product":{"id":1,"name":"Laptop","price":1000},"quantity":2}

C:\week4_lab>docker stop product-service
product-service

C:\week4_lab>curl -X POST http://localhost:5003/orders -H "Content-Type: application/json" -d "{\"product_id\":2,\"quantity\":1}"
{"error":"Product service unavailable"}
```

```
C:\Windows\System32\cmd.e
C:\week4_lab>docker compose logs order-service --follow
time="2026-04-17T01:00:37+02:00" level=warning msg="C:\\week4_lab\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remov
e it to avoid potential confusion"
order-service  |  * Serving Flask app 'app'
order-service  |  * Debug mode: off
order-service  | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
order-service  |  * Running on all addresses (0.0.0.0)
order-service  |  * Running on http://127.0.0.1:5002
order-service  |  * Running on http://172.21.0.3:5002
order-service  | Press CTRL+C to quit
order-service  | 172.21.0.1 - - [16/Apr/2026 22:40:47] "GET /order/1 HTTP/1.1" 404 -
order-service  | 172.21.0.1 - - [16/Apr/2026 22:40:47] "GET /favicon.ico HTTP/1.1" 404 -
order-service  | 172.21.0.1 - - [16/Apr/2026 22:46:34] "POST /orders HTTP/1.1" 415 -
order-service  | 172.21.0.1 - - [16/Apr/2026 22:47:53] "POST /orders HTTP/1.1" 200 -
order-service  | 172.21.0.1 - - [16/Apr/2026 22:50:36] "POST /orders HTTP/1.1" 200 -
order-service  | 172.21.0.1 - - [16/Apr/2026 22:56:04] "POST /orders HTTP/1.1" 503 -
order-service  | 172.21.0.1 - - [16/Apr/2026 22:58:08] "POST /orders HTTP/1.1" 503 -
```

(2) A short report (1-2 pages) answering the reflection questions

1- Which parts of this lab show the benefits of microservices over a monolith?

Benefits of Microservices

- **Independent deployment**: Each service (product-service, order-service) can be updated without affecting the other
- **Scalability**: Services can scale independently (e.g., multiple order-service instances)
- **Separation of concerns**: Each service has a single responsibility
- **Technology flexibility**: Different services could use different technologies

Compared to a monolith, this improves flexibility and maintainability.

2- Which new complexities were introduced by splitting the system into two services?

New Complexities Introduced

Splitting into microservices introduced several challenges:

- **Network communication** between services instead of in-memory calls
- **Service discovery** (handled via Docker DNS)
- **Error handling** for partial failures
- **Debugging difficulty** (logs spread across services)
- **Latency overhead**

This shows microservices are not “free” — they add operational complexity.

3- What would break if network latency increased or one service became slow?

What Breaks with Latency or Slow Service?

If network latency increases or product-service is slow:

- Order requests become **slower or fail**
- Retries increase response time
- System may return **503 errors instead of success**
- User experience degrades significantly

Because communication is **synchronous**, order-service depends directly on product-service.

4- Which 12-factor app principles are visible in this implementation?

Factor App Principles Used

This implementation follows several 12-factor principles:

- **Codebase**: Services are separated logically
- **Dependencies**: Managed via requirements.txt
- **Config**: Environment variables (PRODUCT_SERVICE_URL)
- **Backing services**: product-service acts as a dependency
- **Statelessness**: No persistent state in services
- **Port binding**: Each service exposes its own port

These principles make the system cloud-ready and scalable.